



UNIVERSIDADE ESTADUAL DE CAMPINAS
COLÉGIO TÉCNICO DE LIMEIRA

FARMAGRID+

Plataforma digital integrada para gestão farmacêutica, teleconsultas e acompanhamento clínico

DOCUMENTAÇÃO TÉCNICA E FUNCIONAL — VERSÃO ATUALIZADA CONFORME
O REPOSITÓRIO

Ana Carolina Lanzoni
Heloisa Pola Argentin
Gabriel Andreolli Aires

Limeira
2026

Identificação do trabalho

Trabalho de conclusão de curso apresentado como requisito para obtenção do Diploma de Habilitação Técnica em Desenvolvimento de Sistemas pelo Colégio Técnico de Limeira — COTIL/UNICAMP.

Item	Informação
Projeto	FarmaGrid+
Autores	Ana Carolina Lanzoni; Heloisa Pola Argentin; Gabriel Andreolli Aires
Orientação	Priscila Keli Lima Pinto Frizzarin
Versão desta documentação	2.0 — revisão de aderência ao código
Data da auditoria	22 de setembro de 2026
Base da revisão	Arquivos presentes no repositório local, sem considerar o PDF anterior como especificação obrigatória

Nota de revisão. Esta versão diferencia recursos implementados, parciais/demonstrativos e não implementados. Afirmções de funcionamento foram limitadas ao que é sustentado pelo código-fonte e pelas validações executadas. O PDF anterior foi usado somente como material de comparação.

Sumário

Identificação do trabalho	2
Sumário	3
Resumo	5
1. Introdução	5
1.1 Objetivo geral	5
1.2 Objetivos específicos comprovados no código	5
2. Metodologia da revisão	5
2.1 Escopo inspecionado	6
3. Arquitetura atual	6
3.1 Backend e persistência	6
3.2 Segurança observada	6
4. Matriz consolidada de funcionalidades	7
5. Aplicativo móvel Flutter	9
5.1 Fluxo do paciente	9
5.2 Fluxo do médico	9
5.3 Qualidade do módulo móvel	9
6. Portal web React	9
6.1 Recursos integrados	9
6.2 Limites do portal	10
7. Aplicação desktop Electron	10
7.1 Perfil médico	10
7.2 Gestão de farmácia	10
7.3 Balconista e caixa	10
7.4 Pontos técnicos do desktop	11
8. API REST	11
8.1 Entidades de domínio	11
8.2 Observações de modelagem	11
9. Requisitos funcionais atualizados	12

10. Requisitos não funcionais e realidade atual	12
11. Itens retirados da descrição de funcionalidades concluídas	13
12. Validação executada nesta revisão	13
13. Configuração e execução	14
13.1 API	14
13.2 Aplicativo móvel	14
13.3 Portal web	14
13.4 Desktop	14
14. Riscos, pendências e prioridades	14
15. Conclusão	15
Apêndice A — Inventário resumido de arquivos e evidências	16

Resumo

O FarmaGrid+ é um protótipo acadêmico multiplataforma que integra uma API REST em Java/Spring Boot, um aplicativo Flutter, um portal React/Vite e um cliente desktop Electron. O conjunto implementa autenticação, perfis de paciente e médico, teleconsultas agendadas, prontuários, receitas, exames, farmácias próximas, cupons, estoque por produtos e lotes, controle de validade, funcionários, clientes de farmácia, caixa, vendas, parcerias e relatórios. A solução usa MySQL, JPA, JWT, chamadas HTTP, armazenamento seguro no aplicativo móvel e, no desktop, integrações adicionais com banco direto, geração de PDF/planilhas e IA generativa.

A auditoria também identificou limites relevantes: não existe hardware ou comunicação IoT; não há central de suporte; não existe serviço real de notificações; pagamentos e assinaturas não usam gateway; a videoconferência é feita por link externo; a loja do portal e seu checkout são simulações locais; o cadastro web de farmácia é demonstrativo; e algumas telas móveis utilizam dados estáticos ou fallback. Portanto, a plataforma deve ser apresentada como protótipo funcional em evolução, e não como produto de saúde pronto para produção.

Palavras-chave: saúde digital; gestão farmacêutica; teleconsulta; prontuário eletrônico; prescrição; Flutter; Electron; React; Spring Boot.

1. Introdução

A fragmentação de informações entre pacientes, profissionais de saúde e farmácias dificulta o acompanhamento clínico e a gestão de medicamentos. O FarmaGrid+ propõe centralizar parte desses fluxos em interfaces direcionadas a diferentes perfis: pacientes e médicos no aplicativo móvel, pacientes no portal web e médicos, gestores de farmácia, balconistas e caixas no aplicativo desktop.

O objetivo desta documentação é representar o estado real do projeto. Por isso, conceitos planejados mas sem implementação foram retirados do conjunto de requisitos concluídos ou movidos para a seção de lacunas e trabalhos futuros.

1.1 Objetivo geral

Oferecer um protótipo integrado para cadastro e autenticação de usuários, acompanhamento clínico remoto e operação farmacêutica, compartilhando dados por meio de uma API REST e de um banco MySQL.

1.2 Objetivos específicos comprovados no código

- Cadastrar e autenticar pacientes e médicos, mantendo sessões por token JWT.
- Permitir ao paciente consultar e agendar teleconsultas, visualizar receitas, prontuários e documentos de exame.
- Permitir ao médico consultar pacientes e agenda, registrar prontuários e emitir receitas.
- Apoiar farmácias no cadastro de produtos, lotes, cupons, funcionários, clientes e vendas.
- Gerar indicadores e relatórios operacionais no cliente desktop.
- Disponibilizar interfaces web, móvel e desktop ligadas ao mesmo domínio de negócio.

2. Metodologia da revisão

A revisão percorreu os arquivos-fonte e manifestos dos quatro módulos, incluindo controladores, modelos, repositórios, serviços, telas, configurações, dependências e testes. Arquivos gerados pelas plataformas, imagens e PDFs temporários foram inventariados, mas não foram tratados como prova de funcionalidade.

Cada recurso foi classificado como: **Implementado**, quando há fluxo e persistência/integração coerentes no código; **Parcial ou demonstrativo**, quando há interface ou parte do fluxo, mas falta integração essencial; e **Não implementado**, quando não foi encontrado código correspondente.

2.1 Escopo inspecionado

Módulo	Tecnologia	Papel observado
projetoIntegradorApi	Java 17, Spring Boot 4.0.5, Spring MVC, JPA, Security, MySQL	API REST, autenticação, regras e persistência principal
FarmaGrid+_MOBILE	Flutter/Dart 3.10, HTTP, Secure Storage, Image Picker	Aplicativo para pacientes e médicos
PI_web/farmagrid	React 18, Vite 7	Landing page, cadastro/login e painel do paciente
Desktop/Electron	Electron 38, HTML/CSS/JS, MySQL2, PDFKit, XLSX, Google GenAI	Operação médica e farmacêutica, caixa e relatórios

3. Arquitetura atual

A arquitetura é composta por três clientes e uma API central. O aplicativo Flutter e o portal React apontam, por padrão, para <https://farmagrid.onrender.com>. O Electron também utiliza essa API, mas mantém fluxos que acessam o MySQL diretamente. Essa característica torna o desktop uma aplicação híbrida e aumenta o acoplamento ao esquema do banco.

Origem	Comunicação	Destino
Flutter	HTTPS/JSON com Bearer JWT	API Spring Boot
React	HTTPS/JSON com Bearer JWT; localStorage para token e dados demonstrativos	API Spring Boot e armazenamento local
Electron	IPC entre renderer e processo principal	Handlers do main.js
Electron main	HTTPS/JSON e consultas SQL diretas	API Spring Boot e MySQL
API	JPA/Hibernate	MySQL
Serviços externos	HTTP/SDK	Google Places, Google Meet por link e Gemini quando configurado

3.1 Backend e persistência

A API adota controladores REST, 26 repositórios Spring Data e entidades JPA. O acesso às rotas `/api/**` exige token. Login e cadastro de paciente/médico são públicos. As senhas de login são verificadas e gravadas com BCrypt. O Hibernate está configurado por propriedades externas para conexão com MySQL.

3.2 Segurança observada

- JWT HMAC-SHA256 com validade de oito horas; token transportado no cabeçalho Authorization.
- Sessão móvel armazenada com flutter_secure_storage; portal web mantém token no localStorage.

- CORS e CSRF estão configurados para o modelo stateless da API.
- Limitação crítica: a chave de assinatura JWT está fixa no código-fonte e deve ser migrada para variável de ambiente.
- Limitação: a configuração exige autenticação para /api/**, mas não restringe controladores por papel; um token autenticado pode alcançar rotas de outros perfis se souber os identificadores.
- Limitação: o Electron executa consultas SQL diretas em vários fluxos, contornando parte das regras e controles da API.
- Não há evidência de trilha de auditoria abrangente, criptografia de dados clínicos em repouso, revogação de token, MFA ou gestão formal de consentimento LGPD.

4. Matriz consolidada de funcionalidades

Funcionalidade	Status	Evidência e limite
Cadastro de paciente	Implementado	Web e mobile chamam POST /auth/cadastro/paciente; API persiste paciente e login.
Cadastro de médico	Implementado	Web e mobile chamam POST /auth/cadastro/medico; dados profissionais básicos persistidos.
Cadastro de farmácia	Demonstrativo	No portal web é salvo apenas no localStorage; não há endpoint público equivalente na API.
Login e sessão	Implementado	JWT, BCrypt, restauração de sessão no mobile e token no portal/desktop.
Perfis e fotos	Implementado	Consulta/edição de paciente e médico, foto em base64 e alteração de senha.
Agenda e teleconsulta	Parcial	Agendamento, listagem e reagendamento existem; a sala é um link externo, sem vídeo próprio.
Prontuário	Implementado	Listagem por paciente e inclusão/edição pelo médico; desktop também usa SQL direto.
Receita digital	Implementado	Criação e consulta de prescrições com medicamento, dose, frequência, duração e instruções.
Assinatura/plano	Parcial	Status e dados de assinatura são gravados, mas não há cobrança nem gateway de pagamento.
Exames/documentos	Parcial	Solicitação e armazenamento/consulta de relatório existem; não há laboratório integrado.
Farmácias próximas	Implementado com dependência	Consulta Google Places usando endereço/coordenadas do paciente e chave externa.
Cupons e descontos	Implementado	CRUD no desktop/API, listagem pública autenticada e resgate pelo paciente.
Loja do paciente	Demonstrativo	Portal usa catálogo mock e checkout local; mobile pode cair em dados fallback.
Estoque	Implementado	CRUD de produtos, lotes, quantidade, estoque mínimo e prateleira.
Validade	Implementado	Tela desktop calcula alertas a partir da validade dos lotes e permite remover/resolver lote.
Funcionários	Implementado	Cadastro, edição, listagem e ativação/inativação vinculados à farmácia.
Clientes da farmácia	Implementado	Cadastro e consulta por CPF com vínculo à farmácia.

Funcionalidade	Status	Evidência e limite
Caixa e vendas	Implementado	Carrinho/total/formas de pagamento/troco e persistência de venda; não processa pagamento externo.
Relatórios	Implementado no desktop	Geração e histórico de relatórios de estoque, validade e vendas em PDF/XLSX.
Parcerias médicas	Implementado no desktop	Cadastro, serviços, desconto, status e encaminhamento de paciente; parte usa SQL direto.
Assistente de IA	Parcial	Tela Electron chama Google Gemini quando GEMINI_API_KEY está configurada; não é motor clínico validado.
Notificações	Não implementado	Há opções visuais de preferência, mas não existe serviço de envio, fila ou persistência funcional completa.
Sensor IoT	Não implementado	Não há firmware, protocolo, leitura de sensor ou endpoint de telemetria; a tela sensor.html exibe cupons.
Suporte/Help Center	Não implementado	Não há entidade, tela ou endpoint de tickets.
Auditoria e LGPD	Parcial	Há aceite visual de termos em cadastros, sem registro versionado de consentimento ou trilha abrangente.

5. Aplicativo móvel Flutter

O aplicativo usa a mesma base para os perfis paciente e médico. Após login, o tipo retornado pela API define o fluxo. A sessão é restaurada do armazenamento seguro e as chamadas incluem o token JWT.

5.1 Fluxo do paciente

- Dashboard com dados de perfil e indicadores carregados da API.
- Listagem de teleconsultas e agendamento por médico, data e horário.
- Consulta de receitas e prontuários; botões de visualização/baixar PDF existem na interface, mas nem todo fluxo de download possui integração completa.
- Tela de exames com relatórios existentes e solicitação de novo exame.
- Cupons disponíveis e resgate vinculado ao paciente.
- Farmácias próximas por integração Google Places, condicionada a endereço válido e chave configurada.
- Configuração de perfil, foto, senha, dependentes, cartões e assinatura/plano.
- Loja de medicamentos: tenta consumir produtos, porém o endpoint exige idFarmacia; a tela mantém fallback local e não conclui uma compra real.

5.2 Fluxo do médico

- Painel com contagem de pacientes, consultas e receitas.
- Agenda/listagem de teleconsultas.
- Listagem de pacientes relacionados ao médico e consulta de detalhes.
- Criação e edição de prontuários.
- Emissão de receitas com múltiplos campos clínicos.
- Relatórios/resumo de atividade.
- Configuração de dados pessoais, dados profissionais, preferências, foto e senha.
- Telas de parcerias e insights de IA existem no mobile, mas apresentam conteúdo predominantemente estático; a integração de IA efetiva está no Electron.

5.3 Qualidade do módulo móvel

Foram executados dois testes de widget: abertura na tela de login e estado de carregamento durante restauração de sessão. Ambos passaram. O comando flutter analyze concluiu com 129 apontamentos, entre avisos e informações: código não usado, nomes fora do padrão, APIs depreciadas, uso de contexto após operações assíncronas e outros itens de manutenção. Não foram observados testes de integração com a API.

6. Portal web React

O portal contém landing page responsiva, apresentação de serviços e planos, tela de login/cadastro, confirmação de cadastro, edição de perfil e painel do paciente. O build de produção foi validado com Vite.

6.1 Recursos integrados

- Login de paciente/profissional pela API e armazenamento do token.
- Cadastro de paciente e médico pela API.
- Consulta de teleconsultas e receitas do paciente, enriquecendo consultas com dados dos médicos.
- Agendamento de teleconsulta e abertura do link de sala quando informado.
- Atualização de perfil e foto do paciente, com fallback local em caso de falha no servidor.
- Layout do painel, consultas, receitas, loja e carrinho.

6.2 Limites do portal

O painel implementado é do paciente. Embora o login aceite outros perfis, médico, farmácia, balconista e caixa não recebem painéis web próprios. O cadastro de farmácia é uma demonstração em localStorage. A loja usa catálogo mock e a finalização do carrinho apenas exibe uma confirmação local, pois produtos e vendas exigem vínculo com uma farmácia que o paciente não possui no esquema atual.

7. Aplicação desktop Electron

O desktop reúne a maior parte dos fluxos de operação farmacêutica e também telas para médicos. A navegação é feita por arquivos HTML carregados na janela Electron; o preload expõe funções controladas ao renderer e o processo principal executa chamadas HTTP, consultas MySQL, criação de arquivos e integração de IA.

7.1 Perfil médico

- Dashboard e acesso a prontuários, receitas e teleconsultas.
- Agenda de consultas com criação e reagendamento; entrada em Google Meet por link externo.
- Cadastro e consulta de prontuários e receitas, inclusive fluxo de receita controlada mantido diretamente no desktop/banco.
- Parcerias com cadastro, status, desconto, serviços e encaminhamento de pacientes.
- Configurações pessoais/profissionais, preferências e foto.
- Assistente com Gemini condicionado à variável GEMINI_API_KEY.

7.2 Gestão de farmácia

- Dashboard com indicadores operacionais.
- Produtos e lotes: cadastro, edição, exclusão, quantidade, estoque mínimo, código de barras e prateleira.
- Validade: classificação de lotes por dias restantes e tratamento/remoção de itens.
- Cupons: cadastro, edição, exclusão, status, validade, limite e uso.
- Funcionários: cadastro, edição, consulta e status.
- Relatórios de estoque, validade e vendas, com histórico e download.

7.3 Balconista e caixa

- Cadastro e busca de cliente por CPF, vinculado à farmácia.
- Montagem de venda, cálculo de total e troco, seleção de forma de pagamento e persistência da venda.
- Consulta de vendas com filtros de data.

- Não existe integração com adquirente, PIX, emissão fiscal ou confirmação bancária; as formas de pagamento são registros informativos.

7.4 Pontos técnicos do desktop

Os arquivos JavaScript principais passaram em verificação de sintaxe com Node. O módulo não possui testes automatizados. O package.json declara o script de teste padrão como não implementado. Parte significativa das regras está concentrada em main.js, arquivo extenso que mistura IPC, SQL, API, arquivos e IA; a separação em serviços menores é recomendada.

8. API REST

A API centraliza recursos de autenticação, cadastros, saúde e farmácia. A tabela abaixo agrupa as rotas observadas; todas as rotas /api exigem Bearer JWT pela configuração atual.

Grupo	Operações disponíveis
Autenticação	POST /auth/login; POST /auth/cadastro/paciente; POST /auth/cadastro/medico
Paciente/perfil	CRUD de pacientes; GET/PUT config; PUT foto; assinatura; busca por nome
Médico/perfil	Listagem e consulta; painel; pacientes vinculados; config; dados pessoais/profissionais; preferências; foto
Teleconsultas	Listagem geral/por médico/por paciente; criação; atualização; reagendamento
Prontuários	Listagem, consulta, criação, edição e consulta por paciente
Receitas	Consulta por id, por médico e por paciente; criação
Exames/relatórios	Solicitação de exame; listagem por paciente; upload multipart, download de arquivo e exclusão
Farmácias próximas	Consulta baseada no paciente por meio de Google Places
Produtos/lotos	CRUD de produtos; estoque baixo; listagem/criação/remoção de lotes; alteração de prateleira
Cupons	CRUD por farmácia; disponíveis; listagem e resgate por paciente
Funcionários	Cadastro, consulta, edição, listagem por farmácia e mudança de status
Clientes/vendas	Cadastro/consulta de cliente; listagem e criação de vendas por farmácia
Dependentes/alergias/cartões	CRUDs básicos e consultas vinculadas ao paciente
Disponibilidade médica	Criação, edição, remoção e consulta por médico
Parcerias	Listagem e criação pela API; operações adicionais permanecem no desktop/SQL
Relatórios de farmácia	Consulta, criação e listagem por farmácia

8.1 Entidades de domínio

Foram identificadas entidades para Login, Paciente, Médico, Médico-Clínica, Teleconsulta, Consulta, Disponibilidade Médica, Prontuário, Receita, Solicitação de Exame, Relatório, Produto, Lote, Cupom, Resgate de Cupom, Funcionário, Cliente, vínculo Cliente-Farmácia, Venda, Parceiro, Relatório de Farmácia, Dependente, Alergia, vínculo Paciente-Alergia, Cartão e estruturas auxiliares.

8.2 Observações de modelagem

- Existem modelos paralelos para Consulta e Teleconsulta; consolidá-los reduziria duplicidade.

- O paciente não possui vínculo explícito com farmácia, o que impede determinar a origem de produtos/vendas na loja do paciente.
- Venda armazena os produtos como JSON textual, reduzindo consultas relacionais e integridade referencial.
- O desktop referencia tabelas adicionais para receitas controladas, serviços e encaminhamentos de parceiros que não aparecem como entidades JPA completas.
- Não há migrações versionadas de banco no repositório; o ambiente depende do esquema já existente e do ddl-auto configurado.

9. Requisitos funcionais atualizados

Os requisitos abaixo substituem a lista genérica do documento anterior e refletem o estado observável do projeto.

ID	Requisito	Situação
RF01	Cadastrar pacientes e médicos com validação básica e senha protegida.	Atendido
RF02	Autenticar usuários por e-mail, senha e tipo, emitindo token JWT.	Atendido
RF03	Consultar e atualizar perfis, fotos e senhas de paciente/médico.	Atendido
RF04	Listar, criar e reagendar teleconsultas.	Atendido
RF05	Abrir sala externa associada à consulta quando houver link.	Parcial
RF06	Registrar, editar e consultar prontuários por paciente.	Atendido
RF07	Emitir e consultar receitas digitais estruturadas.	Atendido
RF08	Solicitar exames e armazenar/consultar relatórios anexos.	Parcial
RF09	Gerenciar dependentes, alergias e cartões do paciente.	Atendido no mobile/API
RF10	Localizar farmácias próximas por serviço de mapas.	Condicionado
RF11	Gerenciar assinatura do paciente.	Parcial, sem pagamento
RF12	Gerenciar produtos e lotes por farmácia.	Atendido
RF13	Identificar estoque baixo e vencimentos.	Atendido
RF14	Gerenciar cupons e permitir resgate pelo paciente.	Atendido
RF15	Gerenciar funcionários e seus status.	Atendido
RF16	Cadastrar clientes da farmácia.	Atendido
RF17	Registrar vendas e formas de pagamento.	Atendido sem gateway/fiscal
RF18	Gerar e consultar relatórios operacionais.	Atendido no desktop
RF19	Gerenciar parcerias e encaminhamentos médicos.	Atendido no desktop
RF20	Consultar assistente de IA quando credencial externa estiver configurada.	Parcial/experimental

10. Requisitos não funcionais e realidade atual

Tema	Situação observada	Ação recomendada
Portabilidade	Flutter contém alvos Android, iOS, web, Windows, Linux e macOS; portal é web; Electron é desktop.	Homologar somente plataformas efetivamente testadas.
Segurança	JWT e BCrypt existem, mas segredo fixo e ausência de autorização por perfil são riscos.	Externalizar segredo, aplicar RBAC, validar ownership e revisar logs.
Privacidade	Dados clínicos e pessoais são tratados, sem governança LGPD completa.	Consentimento versionado, minimização, retenção, auditoria e política de exclusão.
Confiabilidade	Há tratamentos de erro e fallbacks, mas poucos testes automatizados.	Adicionar testes unitários, integração com banco isolado e testes ponta a ponta.
Desempenho	Não há teste de carga nem meta mensurável.	Definir SLOs e executar teste de carga da API/banco.
Usabilidade	Interfaces possuem feedback visual e responsividade parcial.	Executar testes com usuários e auditoria de acessibilidade.
Acessibilidade	Não há evidência de conformidade WCAG/semântica completa.	Revisar contraste, foco, teclado, leitores de tela e textos alternativos.
Observabilidade	Uso predominante de console/logs; sem métricas ou rastreamento central.	Adicionar logs estruturados, health checks, métricas e alertas.
Manutenibilidade	Desktop concentra responsabilidades; mobile possui 129 apontamentos estáticos.	Refatorar serviços, resolver warnings e padronizar nomes.

11. Itens retirados da descrição de funcionalidades concluídas

Os itens a seguir constavam ou eram sugeridos no documento anterior, mas não podem ser descritos como entregues no estado atual:

- Sensor inteligente/IoT para leitura automática de estoque, bateria, pareamento ou telemetria.
- Pagamento real de consulta, assinatura ou compra por gateway; emissão de PIX, autorização de cartão e faturamento financeiro.
- Videoconferência nativa com áudio/vídeo; o sistema apenas abre URL de sala/Google Meet.
- Serviço de notificações por push, e-mail ou SMS; as preferências visuais não equivalem ao envio.
- Help Center e gestão de tickets.
- Painel web completo para médico, clínica, farmácia, balconista e caixa.
- Cadastro persistido de clínica e farmácia pelo fluxo público web.
- Validação farmacêutica formal de receita digital, assinatura ICP-Brasil ou QR verificável.
- Integração com laboratório, prontuário nacional, sistema fiscal, adquirente ou operadora de plano.
- Auditoria regulatória completa e comprovação de conformidade LGPD.

12. Validação executada nesta revisão

Verificação	Resultado	Observação
Portal: npm run build	Aprovado	Vite transformou 50 módulos e gerou dist.
Mobile: flutter test	Aprovado	2 testes de widget aprovados.

Verificação	Resultado	Observação
Mobile: flutter analyze	Com apontamentos	129 avisos/informações; sem erro impeditivo de teste.
API: compilação Maven	Aprovada	88 arquivos Java compilados.
API: mvnw test	Bloqueado pelo ambiente	O único teste de contexto requer DB_URL e credenciais MySQL; não há banco de teste isolado.
Electron: node --check	Aprovado	main.js, preload.js, client.js e conexao.js sem erro de sintaxe.
Electron: testes automatizados	Ausentes	Script npm test declara que não há teste especificado.

Os resultados acima verificam compilação, análise estática e testes existentes, mas não comprovam operação ponta a ponta com banco, serviços externos e credenciais de produção.

13. Configuração e execução

13.1 API

- Pré-requisitos: JDK 17 ou superior compatível, MySQL e acesso às dependências Maven.
- Configurar DB_URL, DB_USERNAME e DB_PASSWORD conforme application.properties; configurar chave do Google Places se o recurso for usado.
- Executar no diretório projetoIntegradorApi com mvnw.cmd spring-boot:run no Windows ou ./mvnw spring-boot:run em Unix.
- A API usa a porta configurada pelo ambiente e exige um esquema compatível com as entidades e consultas do desktop.

13.2 Aplicativo móvel

- Instalar Flutter compatível com SDK Dart ^3.10.1.
- Executar flutter pub get e flutter run no diretório FarmaGrid+_MOBILE.
- A URL padrão da API está em lib/config/api_config.dart e aponta para o serviço hospedado.

13.3 Portal web

- Usar Node.js/npm, executar npm ci e npm run dev em PI_web/farmagrid.
- VITE_API_URL pode substituir a URL padrão da API.
- npm run build gera os artefatos de produção em dist.

13.4 Desktop

- Executar npm ci e npm start em Desktop/Electron.
- Configurar API_BASE_URL para trocar a API; configurar credenciais MySQL usadas por src/db/conexao.js.
- Configurar GEMINI_API_KEY somente se o assistente de IA for utilizado.
- Algumas telas dependem simultaneamente da API e do acesso direto ao banco.

14. Riscos, pendências e prioridades

Prioridade	Pendência	Impacto
Crítica	Remover segredo JWT do código, rotacioná-lo e usar variável segura.	Tokens podem ser forjados se o repositório for exposto.
Crítica	Implementar autorização por perfil e propriedade do recurso.	A autenticação isolada não impede acesso indevido entre usuários.
Alta	Eliminar acesso MySQL direto no Electron e mover regras para a API.	Aumenta segurança, consistência e capacidade de auditoria.
Alta	Criar migrações de banco e ambiente de teste isolado.	Reprodutibilidade e testes hoje dependem de infraestrutura externa.
Alta	Definir vínculo paciente-farmácia para a loja e vendas.	Checkout web/mobile permanece simulado ou inconsistente.
Alta	Formalizar proteção de dados de saúde e consentimento LGPD.	Dados sensíveis exigem controles além do protótipo atual.
Média	Consolidar Consulta e Teleconsulta e normalizar itens de venda.	Reduz duplicidade e inconsistência de dados.
Média	Aumentar testes e corrigir apontamentos do Flutter.	Melhora estabilidade e manutenção.
Média	Marcar telas demonstrativas de forma explícita na interface.	Evita confundir protótipo com serviço real.
Baixa	Implementar IoT, notificações e help center apenas após definição de escopo.	Esses recursos não existem e exigem arquitetura própria.

15. Conclusão

O FarmaGrid+ possui uma base funcional ampla e coerente com um projeto integrador: autenticação, três interfaces, domínio clínico, gestão farmacêutica, caixa, relatórios e integrações externas. A API compila, o portal gera build e os testes móveis existentes passam. A documentação anterior, contudo, apresentava como concluídos recursos que não estão no repositório ou estão apenas parcialmente integrados.

Com esta revisão, o projeto passa a ser descrito de forma aderente ao código: teleconsulta significa agendamento mais abertura de link externo; pagamentos são registros sem gateway; IoT e suporte não foram implementados; notificações são apenas opções visuais; e loja/cadastro de farmácia no web contém simulações. Essa distinção preserva os méritos do protótipo e oferece uma base objetiva para a evolução técnica.

Para avançar em direção a um ambiente de produção, as primeiras ações devem ser segurança de JWT e autorização, centralização do acesso a dados na API, migrações e testes isolados, modelagem do vínculo paciente-farmácia e governança de dados sensíveis.

Apêndice A — Inventário resumido de arquivos e evidências

Área	Arquivos/diretórios de referência
API	pom.xml; controller/; model/; repository/; service/; security/; config/; application.properties
Mobile	pubspec.yaml; lib/main.dart; lib/services/; lib/models/; lib/telas/telasPaciente/; lib/telas/telasMedico/
Web	package.json; src/App.jsx; src/pages/; src/dashboards/; src/services/; src/hooks/; src/data/
Desktop	package.json; main.js; preload.js; src/api/client.js; src/db/conexao.js; src/views/*.html
Testes	projetoIntegradorApi/src/test; FarmaGrid+_MOBILE/test; PI_web/farmagrid/src/utlis/maskUtils.test.js

Critério de fechamento: esta documentação registra o repositório observado em 22/09/2026. Funcionalidades adicionadas depois dessa data devem ser novamente confrontadas com código, testes e integrações antes de serem declaradas como concluídas.